Lab 0 - Week 1. Haskell Basics & Monads

Aim

The aim of this assignment is to provide an introduction to coding with Haskell, using Monads and utilising QuickCheck to test Haskell Functions.

This assignment is not graded but you will receive feedback.

Prerequisites

Submission guidelines

Files

- Submit one Haskell file per exercise.
- Name files ExerciseX.hs for exercises and EulerX.hs for Euler problems.
- Each module must be named to match: module ExerciseX where.
- Follow naming conventions exactly — some exercises are auto-graded.
- Do not include personally identifiable information.
- Indicate additional dependencies in a comment at the top of the file.
- Record time spent:
  - Time Spent: X min (in minutes).
- Codegrade: create a new group with all teammates for each assignment.
- Include a txt file containg the report on design choices
- You are not allowed to use GenAI to solve the proposed exercises, but you can use it as a support tool. If you use GenAI, write a comment including how you used it.

Imports

```
import Data.List
import Data.Char
import System.Random
import Test.QuickCheck
```

Useful Functions

```
prime :: Integer -> Bool
prime n = n > 1 && all (\\ x -> rem n x /= 0) xs
    where xs = takeWhile (\\ y -> y^2 <= n) primes
```

```
primes :: [Integer]
primes = 2 : filter prime [3..]
```

Useful logic notation

```
infix 1 -->

(-->) :: Bool -> Bool -> Bool
p --> q = (not p) || q

forall :: [a] -> (a -> Bool) -> Bool
forall = flip all
```

Exercises

Exercise 1

Your programmer Red Curry has written the following function for generating lists of floating point numbers.

```
probs :: Int -> IO [Float]
probs 0 = return []
probs n = do
```

```
    p <- getStdRandom random
    ps <- probs (n-1)
    return (p:ps)
```

He claims that these numbers are random in the open interval (0..1)

Your task is to test whether this claim is correct, by counting the numbers in the quartiles

(0..0.25),[0.25..0.5),[0.5..0.75),[0.75..1)

and checking whether the proportions between these are as expected.

E.g., if you generate 10000 numbers, then roughly 2500 of them should be in each quartile.

Implement this test, and report on the test results.

Deliverables: Test, oral explanation of design choices (in lab), written explanation of design choices, concise test report, indication of time spent

---

Exercise 2

Recognizing triangles

Write a program (in Haskell) that takes a triple of integer values as arguments and gives as output one of the following statements:

- `Not a triangle` if the three numbers cannot occur as the lengths of the sides of triangle,
- `Equilateral` if the three numbers are the lengths of the sides of an equilateral triangle,
- `Rectangular` if the three numbers are the lengths of the sides of a rectangular triangle,
- `Isosceles` if the three numbers are the lengths of the sides of an isosceles (but not equilateral) triangle,
- `Other` if the three numbers are the lengths of the sides of a triangle that is not equilateral, not rectangular, and not isosceles.

Here is a useful datatype definition:

```
data Shape = NoTriangle | Equilateral
       | Isosceles  | Rectangular | Other deriving (Eq,Sho
w)
```

Use the declaration: `triangle :: Integer -> Integer -> Integer -> Shape` with the right properties.

You may wish to consult wikipedia. Indicate how you *tested* or *checked* the correctness of the program.

Deliverables: Haskell program, oral explanation of design choices (in lab), written explanation of design choices, concise test report, indication of time spent.

Exercise 3

The natural number 13 has the property that it is prime and its reversal, the number 31, is also prime. Write a function that finds all primes < 10000 with this property. Follow this type declaration:

```
reversibleStream :: [Integer]
```

To get you started, here is a function for finding the reversal of a natural number:

```
reversal :: Integer -> Integer
reversal = read . reverse . show
```

Implement **at least three** of the following properties and elaborate on each property's coverage. Can you think of any more properties? If so, briefly describe them.

1. Reversal correctness:
    a. This property ensures that the `reversal` function accurately produces the reversal of a number. If this property holds true, it implies that the reversal of a number is a reversible operation, as reversing it twice should yield the original number.

2. Prime reversibility
    a. This property ensures that the function correctly identifies prime numbers that are reversible. If this property holds true, it demonstrates that the combination of the prime-checking function and the reversal-checking function works as intended to identify reversible primes.
3. Prime Membership
    a. This property ensures that the `reversibleStream` function generates a list of prime numbers. If this property holds true for all generated numbers, it guarantees that the function is correctly identifying reversible prime numbers.
4. Reversible Prime Count
    a. This property helps ensure that the `reversibleStream` function is generating the correct number of reversible primes. By comparing the generated count with a pre-computed count, you can verify if the function's output matches expectations.
5. Reversal Symmetry
    a. This property confirms that the function's identification of reversible primes is bidirectional – if a prime number is in the list, its reversal should also be in the list. This property helps validate the correctness of the combination of prime checking and reversal checking.
6. Unique Values
    a. This property confirms that each element in the list is unique
7. Check Maximum Value
    a. This property checks whether the values in the list are less than the maximum value (10000)

Deliverables: Haskell program, oral explanation of design choices (in lab), written explanation of design choices, concise test report, indication of time spent.
